

Task3

1. Introduction This report outlines the design, implementation, and analysis of a C++ program designed to simulate the Aging page replacement algorithm. The program utilizes an 8-bit counter system and reference bits to track page usage over time, efficiently managing memory frames and calculating page hits and faults for a given memory access sequence.

2. Component Analysis

```
struct Frame {  
    int page; //page ID  
    unsigned int counter; //the 8 bit int showing the usage  
    bool referenceBit; //is the page referenced in the recent read  
    bool valid; //true if the slot isnt empty and false if empty  
  
    Frame() : page(-1), counter(0), referenceBit(false), valid(false) {}  
    //valid is false by default because the page should be empty  
};
```

- **Frame Structure:** Represents an individual memory slot. It encapsulates the page ID, an integer counter for the aging process, a referenceBit indicating recent access, and a valid boolean flag to signify if the frame is currently occupied.

```
// check if the page is in the frame  
bool findPage(vector<Frame>& frames, int page, int& hits) {  
    //loop over all the frames  
    for (auto& frame : frames) {  
        //if page not empty and the id is equal (found the page)  
        if (frame.valid && frame.page == page) {  
            frame.referenceBit = true;  
            hits++;  
            return true;  
        }  
    }  
    return false;  
}
```

- **findPage:** Scans the active memory frames for a requested page. If a match is found, it updates the frame's reference bit to true, increments the hit tracker, and returns successfully.

```
// 2.1. The page is not in the frame
void handlePageFault(vector<Frame>& frames, int page, int& faults) {
    faults++;
    int targetIndex = -1;

    // 1. Search for an empty frame
    for (int i = 0; i < (int)frames.size(); i++) {
        if (!frames[i].valid) {
            targetIndex = i;
            break;
        }
    }

    // 2. If no empty frame, find the one with the minimum counter (the oldest accessed frame)
    if (targetIndex == -1) {
        targetIndex = 0;
        for (int i = 1; i < frames.size(); i++) {
            if (frames[i].counter < frames[targetIndex].counter) {
                targetIndex = i;
            }
        }
    }

    // Update the selected frame
    frames[targetIndex].page = page;
    frames[targetIndex].valid = true;
    frames[targetIndex].referenceBit = true;
    frames[targetIndex].counter = 0;
}
```

- **handlePageFault:** Executed when a requested page is absent from memory. It prioritizes filling empty (invalid) frames first. If all frames are occupied, it scans for the frame with the lowest counter value (representing the oldest, least frequently accessed page), evicts it, and loads the new page with an initialized counter of 0.

```

// perform aging logic
void performAging(vector<Frame>& frames) {
    for (auto& frame : frames) {
        if (frame.valid) {
            frame.counter >>= 1; //right shift the counter
            if (frame.referenceBit) {
                //set the leftmost bit to 1
                //or is used to keep the rest of the count as is
                frame.counter |= (1 << 7);
            }
            //set to false
            frame.referenceBit = false;
        }
    }
}

```

- **performAging:** Runs at the end of every page request cycle. It performs a bitwise right-shift ($\gg= 1$) on the counters of all valid frames. If a frame's reference bit was triggered during the cycle, its most significant bit is set to 1 using a bitwise OR operation ($|= (1 \ll 7)$). Finally, all reference bits are reset for the next cycle.

3. Execution Flow

- **Initialization:** The program validates the command-line arguments, parsing the target number of memory frames and the filepath for the input data sequence.
- **File I/O & Processing:** The designated file is read sequentially. A continuous loop extracts requested page IDs one by one until the end of the file is reached.

```

//loop over the file
while (file >> page) {
    cout << "**current memory**\n";
    for (int i = 0; i < frames.size(); i++) {
        if (frames[i].valid)
            cout << frames[i].page << " Age: " << bitset<8>(frames[i].counter) << endl;
    }
    cout << "-----\n";
    cout << "New frame id: " << page << endl;
    cout << "-----\n";
    if (!findPage(frames, page, hits)) {
        handlePageFault(frames, page, faults);
    }
    performAging(frames);
}

```

- **Simulation Cycle:** For each extracted page ID, the program prints the current memory state (displaying active pages and their counters in 8-bit binary). It then attempts to locate the page using findPage. If unsuccessful, it resolves the miss using handlePageFault. Following the memory access attempt, performAging updates the chronological state of all frames.
- **Output:** Upon processing the entire file, the final aggregated counts for successful hits and page faults are printed to the console.

4. Conclusion The code provides a functional and observable simulation of the Aging page replacement algorithm. By printing the binary state of the aging counters at each step, it offers clear insights into how the operating system evaluates memory pages and minimizes fault rates over time.

5. GitHub: [repo](#)

Commit sha: 8d1472115e03cbc92338d73b510ba2fef5b1b295